DATA STRUCTURES & ALGORITHMS

Topics Covered:

- Chapter 1 – Fundamentals of Data Structures & Algorithms
- Chapter 2 – Data Types & Classification
- Chapter 3 – Functions / Methods in Java
- Chapter 4 – Recursion
- Chapter 5 – Java Classes & Objects
- Chapter 6 – Array Data Structure
- Chapter 7 – Sorting Algorithms (Bubble Sort & Radix Sort)
- Chapter 8 – Searching Algorithms (Sequential Search)
- Chapter 9 – Linked List Data Structure
- Chapter 10 – Tree & Binary Tree Data Structures
- Chapter 11 – Comprehensive Q&A Assessment

Chapter 1 – Fundamentals of Data Structures & Algorithms

1.1 What Is a Data Structure?

A data structure is a way of organising and storing data in a computer so that it can be accessed and modified efficiently. Think of it like organising books on a shelf: you could pile them randomly, arrange them alphabetically, or sort them by subject. The arrangement you choose is your 'data structure' and it determines how fast you can find a book.

Real-World Analogy

Phone Contacts App: Every contact is stored in a structured way (name, number, email). When you search for 'Alice', the app uses an efficient data structure to find her instantly rather than reading through every contact one by one.

1.2 What Is an Algorithm?

An algorithm is a step-by-step set of instructions to solve a specific problem. It is like a recipe: a recipe tells a chef exactly what ingredients to use and in what order to cook them. Similarly, an algorithm tells the computer exactly what steps to follow.

Key properties of a good algorithm:

- Correctness – it must produce the correct output for every valid input.
- Efficiency – it should use minimal time and memory.
- Clarity – instructions must be unambiguous.
- Finiteness – it must terminate after a finite number of steps.

Everyday Algorithm Example – Making Tea

Step 1: Boil water. Step 2: Pour water into cup.
Step 3: Add tea bag. Step 4: Wait 3 minutes.
Step 5: Remove tea bag. Step 6: Add sugar/milk as desired.
Step 7: Serve.

Notice: every step is clear, ordered, and the process ends. That is an algorithm!

1.3 Relationship Between Data Structures & Algorithms

Data structures and algorithms are two sides of the same coin. Algorithms manipulate data structures to perform operations like search, insert, update, delete, and sort. In return, a well-chosen data structure makes an algorithm faster.

Data Structure	Typical Algorithm Used On It
Array	Bubble Sort, Linear Search, Radix Sort
Linked List	Insertion, Deletion, Traversal
Tree	Tree Traversal (In/Pre/Post order), Search
Graph	Breadth-First Search, Depth-First Search

1.4 Why Are They Important?

Without efficient data structures and algorithms, programs would be impossibly slow for large data sets. For example:

- Google searches billions of web pages in under one second – made possible by highly optimised data structures (hash tables, trees) and algorithms.
- A hospital database stores millions of patient records and can retrieve any record almost instantly.
- GPS navigation calculates the shortest path between two locations on a map in real time.

Chapter 2 – Data Types & Classification of Data Structures

2.1 What Is a Data Type?

A data type tells the compiler two crucial things:

(1) what kind of value can be stored, and

(2) what operations are allowed on that value.

Think of a data type as a box with a label, the label tells you what you are allowed to put inside.

Data Type	Allowed Values & Operations
int	Whole numbers (e.g. –5, 0, 42). Operations: +, –, ×, ÷, % (modulo), bitwise.
float	Decimal numbers (e.g. 3.14). Operations: +, –, ×, ÷. (No % or bitwise.)
double	Higher-precision decimals. Same as float but wider range.
char	A single character, e.g. 'A'. Operations: comparison, ASCII arithmetic.
boolean	Only true or false. Operations: AND, OR, NOT.
String	Text (sequence of chars). Operations: concatenation, comparison, length.

2.2 Primitive vs Non-Primitive Data Types

Primitive Data Types

Built directly into the programming language. They store exactly ONE value at a time and are the simplest building blocks.

- Examples: int, float, double, char, boolean
- Memory size is fixed and defined by the language.

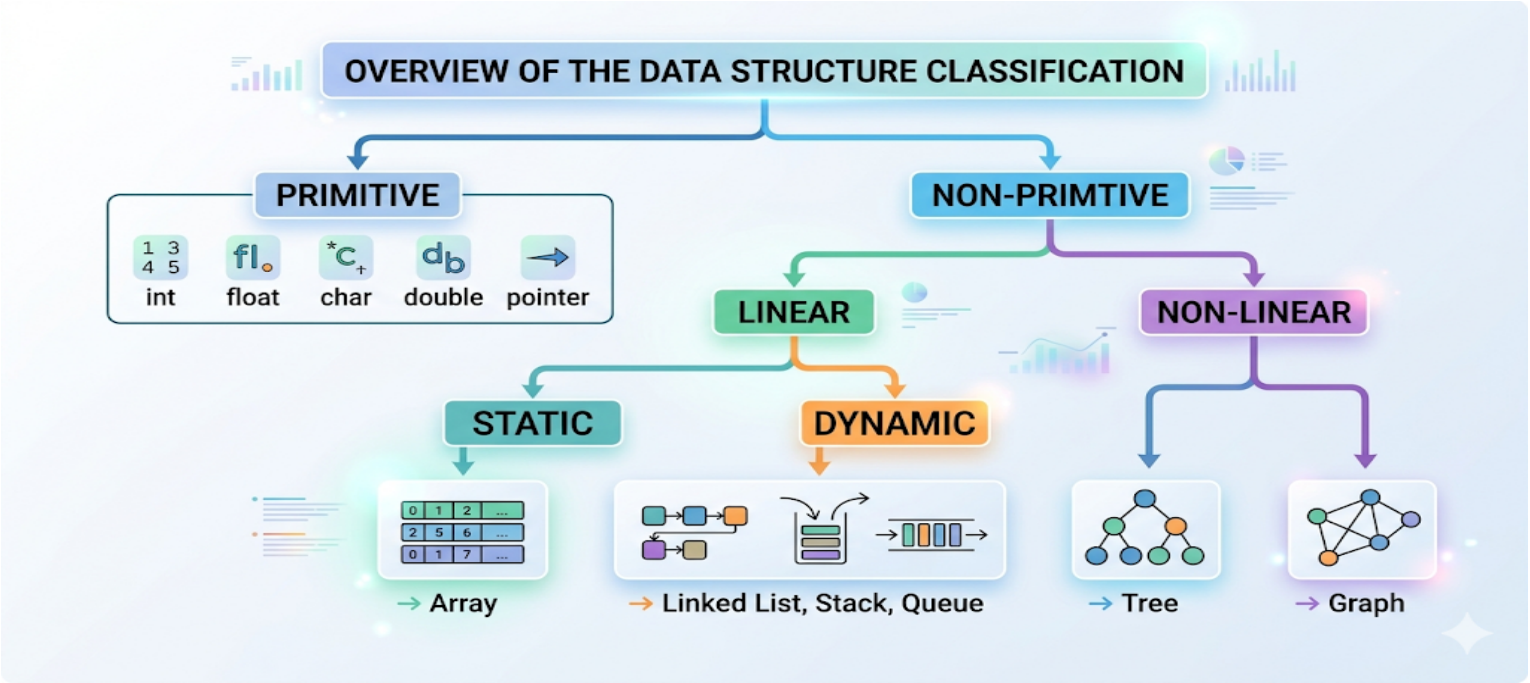
Non-Primitive (Abstract) Data Types

User-defined or library-defined types that can store MULTIPLE values. The programmer (or the system) defines what operations are valid.

- Examples: Arrays, Linked Lists, Stacks, Queues, Trees, Graphs
- Can hold values of different types in a single unit.

2.3 Classification of Data Structures

All data structures fall into two major families:



Linear Data Structures

Elements are arranged in a single, sequential line. Each element (except the first and last) has exactly one predecessor and one successor.

Linear Structure	Key Characteristic
Array (Static)	Fixed size; elements stored in contiguous memory; fast random access by index.
Linked List (Dynamic)	Nodes linked by pointers; size can grow/shrink at runtime.
Stack (Dynamic)	Last In, First Out (LIFO). Think of a stack of plates.
Queue (Dynamic)	First In, First Out (FIFO). Think of a queue at a bus stop.

Non-Linear Data Structures

Elements are arranged in a hierarchical or network structure. You cannot visit all elements in a single sequential pass.

- Tree – hierarchical, like a company org chart or a file system.
- Graph – network of nodes connected by edges, like a road map or social network.

Chapter 3 – Functions / Methods in Java

3.1 What Is a Function (Method)?

A method (Java's term for a function) is a named block of code that performs a specific task. Instead of writing the same code multiple times, you write it once inside a method and call it whenever you need it.

> **Analogy – ATM Machine**
> An ATM has buttons: 'Withdraw', 'Deposit', 'Check Balance'.
> Each button triggers a specific function. You don't need to know HOW the bank transfers money internally – you just call the function (press the button) and get a result.

3.2 Syntax of a Java Method

```
access_specifier  static  return_type  methodName(parameters) {

    // method body – code that does the work

    return value;   // omit if return_type is void
}
```

Keyword Meanings

Keyword	Meaning
public	Accessible from anywhere (outside the class too).
private	Accessible only within the same class.
static	Can be called WITHOUT creating an object of the class.
void	The method returns nothing.
int / float / String …	The method returns a value of that type.
parameters	Input values the method needs to do its work (optional).

3.3 Example – Average of Three Marks

Problem: Write a method that takes three exam marks, calculates the average, and prints it.

```
package CBE_DSA;
import java.util.Scanner;

public class Main {

    // Method to calculate and print the average
    public static void calculateAverage() {
        Scanner input = new Scanner(System.in);

        System.out.println("Enter Mathematics Marks: ");
        float math = input.nextFloat();

        System.out.println("Enter Geography Marks: ");
        float geo = input.nextFloat();

        System.out.println("Enter English Marks: ");
        float engl = input.nextFloat();

        float average = (math + geo + engl) / 3;
        System.out.println("Average Mark is: " + average);
    }

    public static void main(String[] args) {
        calculateAverage();   // Call the method
    }
}
```

Step-by-Step Trace (with marks 90, 80, 70)

1. calculateAverage() is called from main.
2. User enters: 90, 80, 70.
3. average = (90 + 80 + 70) / 3 = 240 / 3 = 80.0
4. Output: Average Mark is: 80.0

Chapter 4 – Recursion

4.1 What Is Recursion?

Recursion is a technique where a method calls itself to solve a smaller version of the same problem. The method keeps calling itself until it reaches a BASE CASE, which stops the recursion.

> **Real-World Analogy – Russian Nesting Dolls (Matryoshka)**
> Open a doll → there is a smaller doll inside → open that → smaller doll → ...
> Eventually you reach the tiniest doll that cannot be opened further.
> That smallest doll is the BASE CASE that stops the recursion.

4.2 Structure of a Recursive Method

```
returnType methodName(parameters) {

    if (BASE CASE condition) {

        return base_value;      // STOP – no more recursion
    }

   else {

        return methodName(smaller_problem);  // RECURSIVE call
    }

}
```

4.3 Example – Factorial

The factorial of a number n (written n!) is: $n! = n \times (n-1) \times (n-2) \times \ldots \times 1$

For example: $5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$

```
public class Main {

    static int factorial(int n) {

        if (n == 1) {          // BASE CASE

           return 1;

        } else {

            return n * factorial(n - 1);  // RECURSIVE call
        }

    }

    public static void main(String[] args) {

        System.out.println("Factorial of 5 is: " + factorial(5));

        // Output: Factorial of 5 is: 120
    }
}
```

How the Recursion Unwinds – Call Stack Trace

Call	Returns
factorial(5)	5 × factorial(4) → 5 × 24 = 120
factorial(4)	4 × factorial(3) → 4 × 6 = 24
factorial(3)	3 × factorial(2) → 3 × 2 = 6
factorial(2)	2 × factorial(1) → 2 × 1 = 2
factorial(1)	BASE CASE → returns 1

WARNING – Infinite Recursion

If you forget the base case, the method calls itself forever and causes a StackOverflowError. Always make sure:

1. There IS a base case.
2. Every recursive call moves TOWARDS the base case.

Chapter 5 – Java Classes & Objects

5.1 What Is a Class?

A class is a blueprint or template for creating objects. It defines what properties (variables) and behaviours (methods) an object will have.

> **Analogy – Cookie Cutter & Cookies**
> The COOKIE CUTTER is the CLASS – it defines the shape.
> Each COOKIE you make from it is an OBJECT – a real thing based on the template.
> All cookies share the same shape (class) but can have different colours/toppings (state).

5.2 What Is an Object?

An object is a real-world entity created from a class. Every object has:

Characteristic	Explanation & Example
State	Properties / attributes of the object. E.g., a Car object has: colour='red', speed=0, wheels=4.
Behaviour	Actions the object can perform. E.g., a Car can accelerate(), brake(), turn().
Identity	A unique name that distinguishes one object from another. E.g., myCar, yourCar.

5.3 Types of Variables in a Class

Variable Type	Description
Instance variable	Declared inside the class but OUTSIDE any method. Not static. Each object gets its own copy.
Static variable	Declared with the 'static' keyword. Shared by ALL objects of the class. Only one copy exists.
Local variable	Declared INSIDE a method. Only accessible within that method. Destroyed when the method ends.

5.4 Constructors

A constructor is a special method that runs automatically when an object is created. It initialises the object's state. Rules:

- Its name MUST match the class name exactly.
- It has NO return type (not even void).
- It CANNOT be abstract, static, final, or synchronized.

Example – Student Class with a Constructor

```
public class Student {

    // Instance variables
    String name;
    int mathMarks;
    int geoMarks;

    // Constructor with parameters
    Student(String n, int m, int g) {
        name      = n;
        mathMarks = m;
        geoMarks  = g;
    }

    // Instance method
    public void displayResults() {
        System.out.println("Student: " + name);
        System.out.println("Maths:   " + mathMarks);
        System.out.println("Geo:     " + geoMarks);
    }

    public static void main(String[] args) {
        Student s1 = new Student("Alice", 85, 72);  // object created
        Student s2 = new Student("Bob",   90, 68);
        s1.displayResults();
        s2.displayResults();
    }
}
```

Chapter 6 – Array Data Structure

6.1 What Is an Array?

An array is a collection of elements of the SAME data type stored in CONTIGUOUS (side-by-side) memory locations. Think of it as a row of numbered boxes, each holding one value.

> **Analogy – Egg Carton**
> An egg carton has 12 numbered slots (index 0 to 11).
> Each slot holds exactly one egg (one value).
> You can instantly reach any egg by its slot number (random access).
> The carton has a FIXED size – you cannot add a 13th slot without a new carton.

6.2 Array Declaration & Initialisation in Java

Step 1 – Declare

```
int data[];       // OR

int[] data;       // OR

int [] data;      // All three are valid
```

Step 2 – Instantiate (allocate memory)

```
data = new int[7];    // Creates 7 slots: data[0] … data[6]
```

Step 3 – Initialise (store values) – 4 Ways

```
// Method 1: Declare, instantiate, then assign one by one
int data[] = new int[7];
data[0] = 100;  data[1] = 234;  data[2] = 100;
data[3] = 4;    data[4] = 19;   data[5] = 100;  data[6] = 345;




// Method 2: Declare + instantiate + initialise in one line
int data[] = new int[]{ 100, 234, 100, 4, 19, 100, 345 };

// Method 3: Shorthand (most common)
int data[] = { 100, 234, 100, 4, 19, 100, 345 };
```

6.3 Working With Arrays – Common Operations

```
int data[] = { 100, 234, 100, 4, 19, 100, 345 };

// Access by index
System.out.println(data[2]);   // Output: 100 (3rd element, index 2)

// Update a value
data[1] = 5000;                // Changes 234 → 5000

// Display ALL elements using a loop
for (int i = 0; i < data.length; i++) {
    System.out.println(data[i]);
}
```

Important: Index vs Position

Indexes START at 0, not 1.

1st element → index 0
2nd element → index 1
3rd element → index 2
nth element → index (n-1)

An array of size 7 has valid indexes 0, 1, 2, 3, 4, 5, 6.
Accessing data[7] would cause an ArrayIndexOutOfBoundsException!

Chapter 7 – Sorting Algorithms

7.1 Bubble Sort

Bubble Sort is the simplest sorting algorithm. It repeatedly compares adjacent elements and swaps them if they are in the wrong order. Larger values 'bubble up' to the end of the array after each pass.

Analogy – Sorting Students by Height

Line up students and compare pairs standing next to each other.
If the left person is taller, they swap positions.
After one full pass, the tallest person has moved to the far right.
Repeat for the remaining students until everyone is sorted.

Step-by-Step Trace on {14, 33, 27, 35, 10}

Pass	Comparison	Action	Array After
Pass 1	14 vs 33	No Swap	[14, 33, 27, 35, 10]
Pass 1	33 vs 27	SWAP 33 & 27	[14, 27, 33, 35, 10]
Pass 1	33 vs 35	No Swap	[14, 27, 33, 35, 10]
Pass 1	35 vs 10	SWAP 35 & 10	[14, 27, 33, 10, 35]
Pass 2	14 vs 27	No Swap	[14, 27, 33, 10, 35]
Pass 2	27 vs 33	No Swap	[14, 27, 33, 10, 35]
Pass 2	33 vs 10	SWAP 33 & 10	[14, 27, 10, 33, 35]
Pass 3	14 vs 27	No Swap	[14, 27, 10, 33, 35]
Pass 3	27 vs 10	SWAP 27 & 10	[14, 10, 27, 33, 35]
Pass 4	14 vs 10	SWAP 14 & 10	[10, 14, 27, 33, 35]
DONE	—	Sorted!	[10, 14, 27, 33, 35]

Java Implementation

```java
public class JavaLoopDemo{
    Run | Debug
    public static void main(String[] args) {
        // Array - with your 5 specific numbers
        int arr[] = new int[]{16, 4, 2, 10, 6};

        for(int i=0; i <= 3; i++) {

            // Inner loop compares adjacent elements
            for(int j=0; j <= 3; j++) {
                if(arr[j] > arr[j+1]) {

                    // Swap logic using a temporary variable
                    int temp = arr[j];
                    arr[j] = arr[j+1];
                    arr[j+1] = temp;
                }
            }
        }

        // Output Loop - Updated to print all 5 elements
        for(int i=0; i <= 4; i++) {
            System.out.print(arr[i] + (""));
        }
    }
}
```

Output

```
2 4 6 10 16
```

7.2 Radix Sort

Radix Sort sorts integers digit by digit, starting from the LEAST significant digit (units place) to the MOST significant digit. It uses 10 buckets (0–9), one for each possible digit value.

Analogy – Sorting Library Cards

Imagine sorting thousands of numbered library cards.
Round 1: Sort by the LAST digit only (units place).
Round 2: Re-sort by the SECOND-TO-LAST digit (tens place).
Round 3: Re-sort by the THIRD digit (hundreds place).
After enough rounds, the cards are fully sorted!

Step-by-Step Trace on {132, 543, 783, 063, 007, 898, 049}

Pass 1 – Sort by UNITS digit (last digit):

Bucket	Numbers Placed In It
Bucket 2	132
Bucket 3	543, 783, 063
Bucket 7	007
Bucket 8	898
Bucket 9	049

Collected in order: 132, 543, 783, 063, 007, 898, 049

Pass 2 – Sort by TENS digit:

Bucket	Numbers Placed In It
Bucket 0	007
Bucket 3	132
Bucket 4	543, 049
Bucket 6	063
Bucket 8	783, 898

Collected in order: 007, 132, 543, 049, 063, 783, 898

Pass 3 – Sort by HUNDREDS digit:

Bucket	Numbers Placed In It
Bucket 0	007, 049, 063
Bucket 1	132
Bucket 5	543
Bucket 7	783
Bucket 8	898

Final Sorted Array: 007, 049, 063, 132, 543, 783, 898

Chapter 8 – Sequential (Linear) Search Algorithm

8.1 What Is a Search Algorithm?

A search algorithm locates a specific element (called the KEY or TARGET) within a data collection. Sequential Search is the simplest: it checks each element one by one from the beginning until the target is found or the list is exhausted.

Analogy – Looking for a Friend in a Crowd

You enter a dark room full of people. You have no idea where your friend is.
You walk up to the first person: 'Are you Ali?' – No.
Second person: 'Are you Ali?' – No.
You continue until you find Ali or reach the last person.
That is sequential search!

8.2 Algorithm Steps

1. Declare an array A and store the elements.
2. Declare variable K = the value to search for.
3. Loop through each element: compare A[i] with K.
4. If A[i] == K: return 1 (found!). Stop searching.
5. If the loop finishes without a match: return -1 (not found).
6. Display an appropriate message to the user.

8.3 Java Implementation

```
public class Main {

    // Returns 1 if K is found in A; returns -1 otherwise
    static int findValue(int A[], int K) {
        for (int i = 0; i < A.length; i++) {
            if (A[i] == K) {
                return 1;    // FOUND
            }
        }
        return -1;           // NOT FOUND
    }

    public static void main(String[] args) {
        int A[] = { 10, 50, 30, 70, 80, 60, 20, 90, 40 };
        int K = 20;

        if (findValue(A, K) == 1) {
            System.out.println("Element " + K + " was FOUND.");
        } else {
            System.out.println("Element " + K + " was NOT found.");
        }
        // Output: Element 20 was FOUND.
    }
}
```

Trace – Searching for 20 in {10, 50, 30, 70, 80, 60, 20, 90, 40}

Index i	A[i] == 20?
0	A[0]=10 → No
1	A[1]=50 → No
2	A[2]=30 → No
3	A[3]=70 → No
4	A[4]=80 → No
5	A[5]=60 → No
6	A[6]=20 → YES! Return 1. FOUND!

Chapter 9 – Linked List Data Structure

9.1 What Is a Linked List?

A linked list is a linear data structure made of NODES. Unlike arrays, nodes are NOT stored in contiguous memory. Instead, each node contains:

- DATA – the actual value stored.
- LINK (Pointer/Reference) – the memory address of the NEXT node.

The last node points to NULL, signalling the end of the list.

Analogy – Treasure Hunt

Each clue (node) in a treasure hunt tells you: (1) a piece of the message (data), and (2) where the NEXT clue is hidden (link/pointer).
You follow the chain of clues until you reach the treasure (NULL = end).

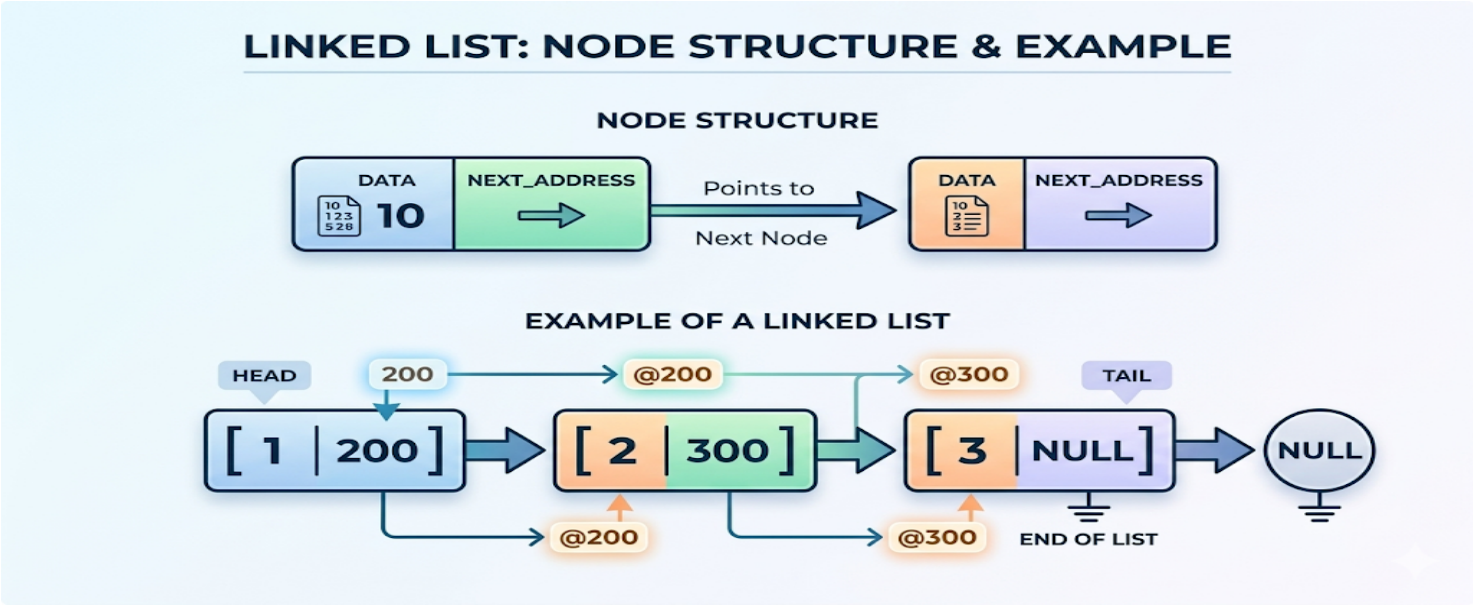
9.2 Why Use a Linked List Instead of an Array?

Array Limitation	How Linked List Solves It
Fixed size – must declare size in advance.	Dynamic size – nodes are added/removed at runtime.
Inserting an element in the middle requires shifting all subsequent elements.	Inserting a node only requires updating two pointers.
Memory is allocated as one contiguous block – wasted if not fully used.	Memory is allocated node by node – no wasted space.

9.3 Types of Linked Lists

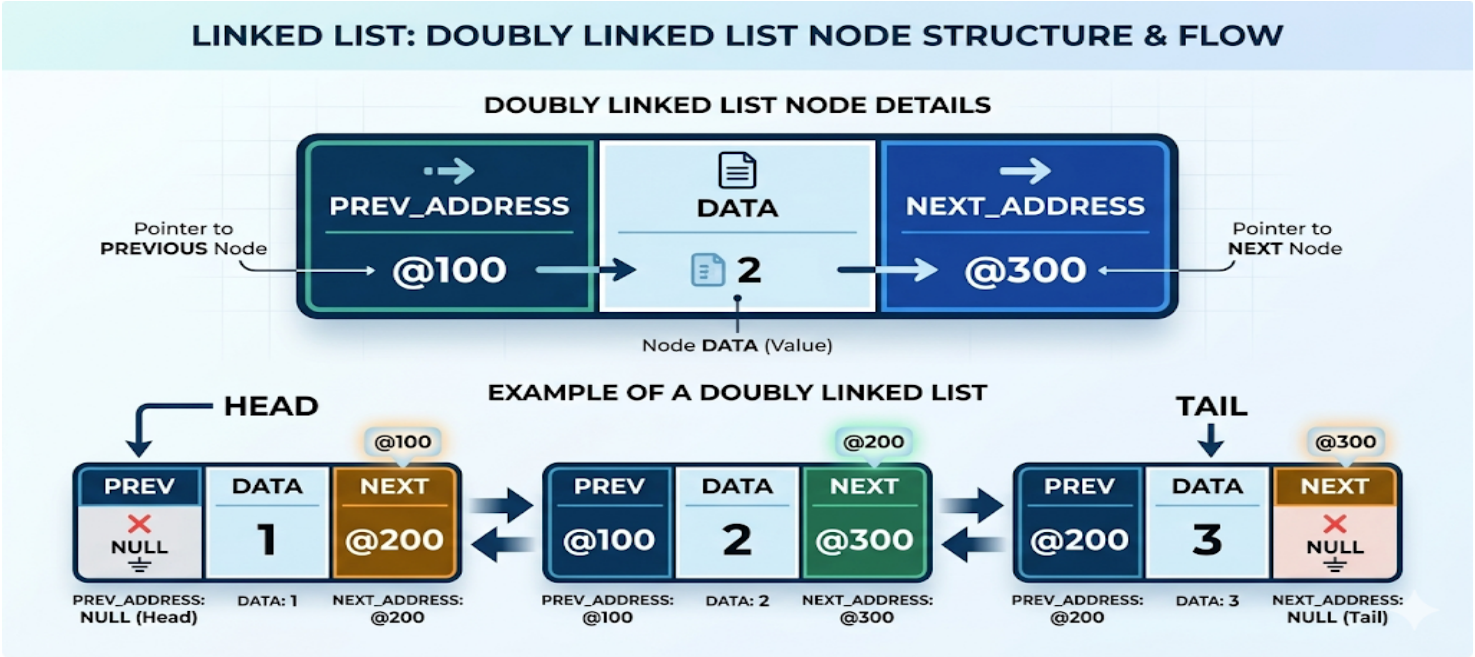
1. Singly Linked List

Each node has one link pointing to the NEXT node only. Traversal is one-directional (forward only).



2. Doubly Linked List

Each node has TWO links: one pointing to the PREVIOUS node and one to the NEXT node. Traversal is bidirectional (forward AND backward).

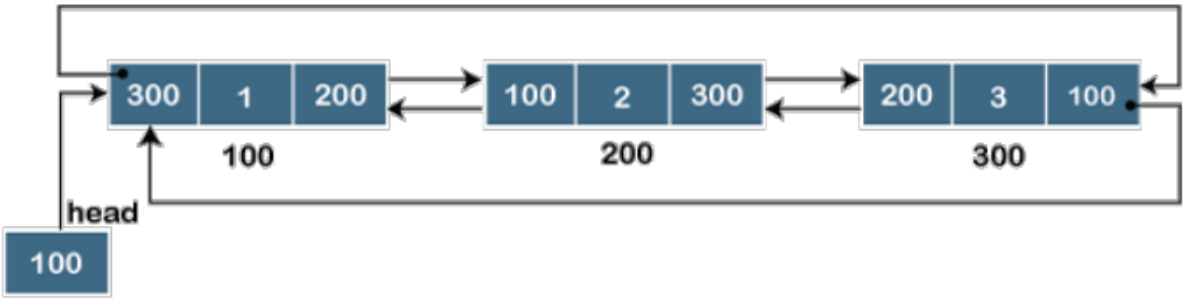


3. Circular Singly Linked List

Like a singly linked list, but the LAST node points back to the FIRST node (instead of NULL).
Useful for round-robin scheduling (e.g., multiplayer game turns).

4. Circular Doubly Linked List

Like a doubly linked list, but the last node's NEXT points to the first node, and the first node's PREV points to the last node. There is no NULL.



9.4 Operations on a Linked List

Operation	Description
Insertion	Add a new node (at the beginning, middle, or end of the list).
Deletion	Remove an existing node by unlinking it from the chain.
Display / Traversal	Visit each node from head to tail and print its data.
Search	Traverse the list comparing each node's data to the search key.

Chapter 10 – Tree & Binary Tree Data Structures

10.1 What Is a Tree?

A tree is a NON-LINEAR, HIERARCHICAL data structure. Elements (nodes) are arranged in parent-child relationships across multiple levels – just like a family tree or a company organisational chart.

Analogy – Family Tree
Your grandfather is at the TOP (root).
His children are on the next level (child nodes).
Their children are another level down (grandchildren = grandchild nodes).
People with no children are at the bottom (leaf nodes).
You cannot reach a grandchild without going through their parent first.

10.2 Key Terminology

Term	Definition
Root Node	The topmost node; the only node with no parent. (E.g., node A)
Parent Node	A node that has one or more children. (E.g., B is parent of D and E)
Child Node	A node that has a parent. (E.g., D and E are children of B)
Leaf / External Node	A node with NO children. (E.g., K, L, M, N, O, P)
Siblings	Nodes that share the same parent. (E.g., D and E are siblings)
Ancestor	Any node on the path from a given node up to the root.
Descendant	Any node on the path from a given node down to any leaf.
Subtree	A node and all its descendants.
Depth of a node	Number of edges from the root to that node.
Height of a node	Number of edges on the longest path from that node to a leaf.
Height of the tree	Height of the root node.
Edge	The connection (link) between two nodes.

10.3 Properties of a Tree

- A tree with N nodes has exactly N–1 edges.
- There is exactly ONE path between any two nodes.
- Depth of root = 0; height of a leaf node = 0.

10.4 Applications of Trees

- File systems – folders and files form a tree (Windows Explorer, Linux directories).
- Database indexing – B-Trees are used to speed up queries in relational databases.
- HTML/XML documents – the DOM (Document Object Model) is a tree structure.
- Network routing – routers use spanning trees to avoid loops.
- Decision making in AI – decision trees power many machine-learning models.

10.5 Binary Tree

A Binary Tree is a tree where EVERY NODE has AT MOST 2 children: a LEFT child and a RIGHT child.

Properties of a Binary Tree

- At level i, the maximum number of nodes = 2^i.
- Maximum total nodes at height h = 2^(h+1) - 1.
- Number of Levels L = h + 1, so max nodes = 2^L - 1.
- Minimum nodes → maximum height (like a straight line/degenerate tree).
- Maximum nodes → minimum height (complete tree).

10.6 Types of Binary Trees

Type	Rule
Full / Strict Binary Tree	Every node has EXACTLY 0 or 2 children. No node can have just 1 child.
Complete Binary Tree	All levels are completely filled EXCEPT possibly the last level, which is filled left to right.
Perfect Binary Tree	All internal nodes have exactly 2 children AND all leaf nodes are at the SAME level.
Degenerate (Skewed) Tree	Every internal node has exactly 1 child. Looks like a linked list. Very inefficient.
Balanced Binary Tree	Left and right subtree heights differ by AT MOST 1. Examples: AVL Tree, Red-Black Tree.

Quick Visual Reference

Binary Tree Type	Shape Description
Full BT	Triangle shape – symmetric, every internal node has 2 kids.
Complete BT	Triangle with the bottom row filled from LEFT to RIGHT.
Perfect BT	Symmetric triangle – ALL leaf nodes at the same level.
Degenerate BT	Straight line (like a linked list) – zigzagging left or right.
Balanced BT	Roughly equal left and right sides at every level.

Chapter 11 – Comprehensive Assessment Q&A

This chapter tests your understanding of all topics covered. It includes six question types. Answers are provided at the end of each section.

Section A – Match the Items

Draw a line (or write the letter) matching each term in Column A with its correct description in Column B.

Column A – Term	Column B – Description	
1. Array	A. A sorting method that groups digits by place value.	___
2. Recursion	B. The topmost node of a tree with no parent.	___
3. Stack	C. A node that has no children.	___
4. Bubble Sort	D. A method that calls itself until a base case is reached.	___
5. Queue	E. Static data structure; elements stored contiguously in memory.	___
6. Root Node	F. FIFO structure – first in, first out.	___
7. Leaf Node	G. A blueprint used to create objects in Java.	___
8. Radix Sort	H. Compares adjacent pairs and swaps them if out of order.	___
9. Class	I. LIFO structure – last in, first out.	___
10. Linked List	J. Nodes connected by pointers; dynamic size; stored non-contiguously.	___

Answers – Section A

1→E 2→D 3→I 4→H 5→F 6→B 7→C 8→A 9→G 10→J

Section B – Multiple Choice Questions

Circle the letter of the BEST answer.

1. What is the index of the FIRST element in a Java array?

A) 1
B) 0
C) -1
D) Depends on the array type
Answer: B

2. Which data structure follows the LIFO (Last In, First Out) principle?

A) Queue
B) Linked List
C) Stack
D) Array
Answer: C

3. What happens when a recursive method has no base case?

A) It returns null
B) It runs once then stops
C) It causes a StackOverflowError
D) It compiles but produces 0
Answer: C

4. How many edges does a tree with 8 nodes have?

A) 8
B) 7
C) 9
D) 6
Answer: B

5. Which sorting algorithm uses 10 buckets (0–9) and processes digits from least to most significant?

A) Bubble Sort
B) Selection Sort
C) Radix Sort
D) Insertion Sort
Answer: C

6. In a doubly linked list, each node contains:

A) Only data and a next pointer
B) Data, a next pointer, and a previous pointer
C) Only data
D) Two data fields and one pointer
Answer: B

7. What is the maximum number of nodes at level 3 of a binary tree?

A) 4
B) 6

C) 8
D) 16
Answer: C

8. A constructor in Java:
A) Must return an int value
B) Must be declared static
C) Must have the same name as the class
D) Can have any name
Answer: C

9. Which of the following is a NON-linear data structure?
A) Array
B) Stack
C) Queue
D) Tree
Answer: D

10. In sequential search, what value is returned if the target element is NOT found?
A) 0
B) null
C) -1
D) false
Answer: C

11. Which variable type in Java is shared by ALL objects of a class?
A) Local variable
B) Instance variable
C) Static variable
D) Parameter variable
Answer: C

12. An array is an example of a _____ data structure.
A) Dynamic
B) Non-linear
C) Static
D) Tree-based
Answer: C

13. In Bubble Sort on an array of n elements, how many passes are needed in the worst case?
A) 1
B) n
C) n-1
D) n/2
Answer: C

14. What does 'head' refer to in a linked list?
A) The last node
B) The NULL pointer
C) The first node

D) Any random node
Answer: C

15. A Perfect Binary Tree at height h has a total of _____ nodes.

A) 2h
B) 2^(h+1)
C) 2^(h+1) - 1
D) h + 1
Answer: C

Section C – True or False

Write TRUE or FALSE in the space provided.

No.	Statement	Answer
Part 1 – General Concepts		
1.	An array in Java can store elements of different data types.	FALSE
2.	A data structure is a way of organising data so it can be efficiently used.	TRUE
3.	Algorithms and data structures are independent of each other.	FALSE
4.	A primitive data type can store more than one value at the same time.	FALSE
5.	Non-primitive data types are also called abstract data types.	TRUE
6.	Linear data structures store elements in a sequential order.	TRUE
7.	Trees and Graphs are examples of linear data structures.	FALSE
8.	A static data structure has a fixed size that cannot change at runtime.	TRUE
9.	A dynamic data structure can grow or shrink in size during execution.	TRUE
10.	The 'float' data type supports the modulo (%) operator.	FALSE
Part 2 – Functions & Methods		
11.	A method declared as 'void' returns a value to the caller.	FALSE
12.	A static method can be called without creating an object.	TRUE
13.	A method's parameter variables are accessible outside the method.	FALSE
14.	The 'public' keyword means the method is accessible only within its own class.	FALSE
15.	A Java method can call another Java method.	TRUE
16.	The return type of a constructor is 'void'.	FALSE
17.	A method with return type 'int' must contain a 'return' statement.	TRUE
18.	You can have more than one method with the same name in a class (method overloading).	TRUE
19.	Local variables are created when a method is called and destroyed when it ends.	TRUE
20.	Instance methods can be called directly by static methods without an object.	FALSE
Part 3 – Recursion		
21.	Recursion requires a base case to prevent infinite loops.	TRUE

22.	A recursive method always runs faster than an equivalent loop.	FALSE
23.	In factorial(5), the method calls itself with factorial(4).	TRUE
24.	The factorial of 1 is 0.	FALSE (it is 1)
25.	Every recursive call moves the problem closer to the base case.	TRUE
26.	Forgetting the base case causes a StackOverflowError.	TRUE
27.	A method that calls itself is called an iterative method.	FALSE (recursive)
28.	Recursion makes code compact but can be harder to understand.	TRUE
29.	The factorial of 5 is 120.	TRUE
30.	Recursion can only be used with integer numbers.	FALSE
Part 4 – Classes & Objects		
31.	An object's constructor has the same name as the class.	TRUE
32.	A constructor must have an explicit return type such as 'void'.	FALSE
33.	A static variable is shared by all objects of a class.	TRUE
34.	An instance variable gets memory at compile time.	FALSE (runtime)
35.	A class is a blueprint for creating objects.	TRUE
36.	An object can only have one state at a time.	FALSE
37.	Declaring an object using 'new ClassName()' is called instantiating.	TRUE
38.	A local variable can be accessed from anywhere in the program.	FALSE
39.	A Java class must contain a main() method to be valid.	FALSE
40.	Static methods cannot directly access instance variables without an object.	TRUE
Part 5 – Arrays		
41.	Non-primitive data types can store more than one value.	TRUE
42.	Array elements are stored in contiguous memory locations.	TRUE
43.	An array of size 5 has valid indexes from 1 to 5.	FALSE (0 to 4)
44.	You can change the size of a Java array after it is created.	FALSE
45.	The keyword 'new' is used to allocate memory for an array.	TRUE
46.	Accessing data[10] in an array of size 10 causes an error.	TRUE
47.	Java allows you to store integers and strings in the same array.	FALSE

48.	int data[] = {1,2,3}; declares, instantiates, and initialises the array in one line.	TRUE
49.	arr.length gives the index of the last element.	FALSE (gives total size)
50.	A for-loop is commonly used to traverse all elements of an array.	TRUE
Part 6 – Sorting Algorithms		
51.	In Bubble Sort, smaller elements bubble up to the beginning of the array.	FALSE (larger to end)
52.	Radix Sort processes digits starting from the most significant digit.	FALSE (least first)
53.	Bubble Sort compares adjacent pairs of elements.	TRUE
54.	Radix Sort uses exactly 10 buckets (0–9) for decimal integers.	TRUE
55.	Bubble Sort is a comparison-based sorting algorithm.	TRUE
56.	After the first pass of Bubble Sort, the smallest element is at index 0.	FALSE (largest at end)
57.	Radix Sort can sort negative numbers without any modification.	FALSE
58.	A temporary variable (temp/K) is needed to swap two array elements.	TRUE
59.	Bubble Sort and Radix Sort always produce the same sorted output for the same input.	TRUE
60.	Radix Sort is classified as a linear (non-comparison-based) sorting algorithm.	TRUE
Part 7 – Searching Algorithms		
61.	Sequential search starts searching from one end and moves through each element.	TRUE
62.	In sequential search, the algorithm always checks every element in the array.	FALSE (stops when found)
63.	Sequential search requires the array to be sorted first.	FALSE
64.	Sequential search returns 1 if the target element is found.	TRUE
65.	Sequential search is the most efficient searching algorithm for very large	FALSE

	arrays.	
Part 8 – Linked Lists		
66.	The size of a linked list must be declared before use.	FALSE
67.	Linked list nodes are stored in contiguous memory locations.	FALSE (non-contiguous)
68.	In a singly linked list, each node has a pointer to the next node only.	TRUE
69.	In a doubly linked list, traversal can be done in both directions.	TRUE
70.	The last node in a singly linked list points to NULL.	TRUE
71.	A circular linked list's last node points to the first node.	TRUE
72.	Inserting a node into a linked list is faster than inserting into an array (middle insertion).	TRUE
73.	Linked lists support random access by index just like arrays.	FALSE
74.	A doubly linked list node has three fields: previous pointer, data, and next pointer.	TRUE
75.	Linked lists waste memory if not fully utilised because of their fixed allocation.	FALSE (arrays waste; linked lists allocate as needed)
Part 9 – Trees & Binary Trees		
76.	A leaf node in a tree has no children.	TRUE
77.	The root node of a tree has a depth of zero.	TRUE
78.	A complete binary tree must have all leaf nodes at the same level.	FALSE
79.	A tree data structure is non-linear.	TRUE
80.	In a binary tree, every node can have at most 2 children.	TRUE
81.	A degenerate binary tree is shaped like a linked list.	TRUE
82.	A tree with 10 nodes has 10 edges.	FALSE (9 edges: N-1)
83.	In a perfect binary tree, all leaf nodes are at the same level.	TRUE

84.	A balanced binary tree allows the left and right subtrees to differ by more than 1.	FALSE (at most 1)
85.	The height of the root node equals the height of the entire tree.	TRUE
86.	Siblings are nodes that share the same parent.	TRUE
87.	You can navigate to a child node without traversing its parent in a tree.	FALSE
88.	A full binary tree requires every node to have exactly 0 or 2 children.	TRUE
89.	AVL trees and Red-Black trees are examples of balanced binary trees.	TRUE
90.	File systems stored on a hard drive are an example of a tree data structure.	TRUE

Section D – Fill in the Blanks

Complete each sentence with the correct word or phrase.

Part 1 – General Concepts (Questions 1–15)

1. A _____ is a step-by-step procedure for solving a problem.
   ✔ Answer: Algorithm
2. In Java, an array index starts at _____.
   ✔ Answer: 0
3. A recursive method must have a _____ to stop infinite calls.
   ✔ Answer: base case
4. The _____ node is the topmost node of a tree.
   ✔ Answer: root
5. A _____ linked list has its last node pointing back to the first node.
   ✔ Answer: circular
6. In Bubble Sort, two adjacent elements are _____ if they are out of order.
   ✔ Answer: swapped
7. A _____ data type stores only one value (e.g., int, float).
   ✔ Answer: primitive
8. In a doubly linked list, each node stores the address of both the _____ and the _____ node.
   ✔ Answer: previous / next
9. The height of a leaf node is _____.
   ✔ Answer: 0
10. A _____ Binary Tree has all internal nodes with exactly 0 or 2 children.
    ✔ Answer: Full / Strict
11. A Java _____ is called automatically when an object is created using 'new'.
    ✔ Answer: constructor
12. Sequential search returns _____ if the element is not found.
    ✔ Answer: -1
13. In Radix Sort, we use _____ buckets numbered 0 through 9.
    ✔ Answer: 10
14. A _____ variable in Java can only be accessed within the method it is declared in.
    ✔ Answer: local
15. Non-linear data structures include _____ and _____.
    ✔ Answer: trees / graphs

Part 2 – Data Types & Classification (Questions 16–22)

16. A non-primitive data type is also called an _____ data type.
    ✔ Answer: abstract

17. The two main categories of data structures are _____ and _____ data structures.
   ✔ Answer: primitive / non-primitive
18. A _____ data structure has a fixed size that cannot change at runtime.
   ✔ Answer: static
19. A _____ data structure can grow or shrink in size during program execution.
   ✔ Answer: dynamic
20. Arrays belong to the _____ category of linear data structures.
   ✔ Answer: static
21. A data type defines the _____ of values that can be stored and the _____ allowed on them.
   ✔ Answer: domain / operations
22. The 'float' data type does NOT support the _____ operator (%).
   ✔ Answer: modulo

Part 3 – Functions & Methods (Questions 23–27)

23. A method declared with the keyword _____ does not return any value.
   ✔ Answer: void
24. A _____ method can be called without creating an object of the class.
   ✔ Answer: static
25. The keyword _____ means the method or variable can be accessed from outside the class.
   ✔ Answer: public
26. Values passed into a method are called _____ or arguments.
   ✔ Answer: parameters
27. A method that returns a decimal value would use _____ as its return type.
   ✔ Answer: float / double

Part 4 – Recursion (Questions 28–32)

28. In the factorial program, factorial(1) returns _____ because it is the base case.
   ✔ Answer: 1
29. Each time a recursive method calls itself, a new _____ frame is added to memory.
   ✔ Answer: stack
30. Forgetting the base case in a recursive method causes a _____.
   ✔ Answer: StackOverflowError
31. The factorial of 0 (0!) is defined as _____.
   ✔ Answer: 1
32. Recursion makes code _____ but can be more complex to understand.
   ✔ Answer: compact

Part 5 – Classes & Objects (Questions 33–38)

33. A class is a _____ or blueprint from which objects are created.
    ✔ Answer: template
34. An object's _____ describes what actions it can perform (represented by methods).
    ✔ Answer: behaviour
35. An object's _____ describes its properties (represented by variables).
    ✔ Answer: state
36. A _____ variable is declared inside the class but outside any method, without 'static'.
    ✔ Answer: instance
37. Declaring an object using 'new ClassName()' is also called _____ a class.
    ✔ Answer: instantiating
38. Instance methods can be accessed from the main method using an _____.
    ✔ Answer: object

Part 6 – Arrays (Questions 39–45)

39. All elements in a Java array are stored in _____ memory locations.
    ✔ Answer: contiguous
40. An array of size 10 has valid indexes from 0 to _____.
    ✔ Answer: 9
41. The keyword _____ is used to allocate memory for a Java array.
    ✔ Answer: new
42. To find the number of elements in an array named 'arr', you use _____.
    ✔ Answer: arr.length
43. Accessing an index beyond the array's size causes an _____.
    ✔ Answer: ArrayIndexOutOfBoundsException
44. A _____ array stores elements in a single row using one index.
    ✔ Answer: single-dimensional
45. A _____ array uses two indexes (row and column) to store elements in a grid.
    ✔ Answer: multi-dimensional

Part 7 – Sorting Algorithms (Questions 46–50)

46. Bubble Sort is a _____ -based algorithm because it uses element comparisons to decide order.
    ✔ Answer: comparison
47. In Bubble Sort, the _____ element moves to its correct position after each pass.
    ✔ Answer: largest
48. Radix Sort processes digits starting from the _____ significant digit.
    ✔ Answer: least
49. Radix Sort is classified as a _____ sorting algorithm (not comparison-based).

✔️ Answer: linear / non-comparison

50. In Bubble Sort, a temporary variable (often called 'K' or 'temp') is used to _____ two elements.

✔️ Answer: swap

Part 8 – Searching (Questions 51–53)

51. Sequential Search is also known as _____ Search.

✔️ Answer: Linear

52. In the best case, Sequential Search finds the target at index _____ (first position).

✔️ Answer: 0

53. Sequential Search is the _____ searching algorithm — simple but potentially slow on large arrays.

✔️ Answer: easiest / simplest

Part 9 – Linked Lists (Questions 54–60)

54. Each node in a linked list contains two parts: the _____ and the _____.

✔️ Answer: data / link (address)

55. The last node in a singly linked list points to _____.

✔️ Answer: NULL

56. The first node in a linked list is called the _____.

✔️ Answer: head

57. A _____ linked list allows traversal in both forward and backward directions.

✔️ Answer: doubly

58. Linked lists store nodes in _____ memory locations (not side by side).

✔️ Answer: non-contiguous / random

59. The four basic operations on a linked list are insertion, deletion, display, and _____.

✔️ Answer: search

60. Unlike arrays, linked lists do not require the programmer to define their _____ in advance.

✔️ Answer: size

Part 10 – Trees & Binary Trees (Questions 61–70)

61. A tree data structure is _____ because elements are not stored sequentially.

✔️ Answer: non-linear

62. Nodes that share the same parent are called _____.

✔️ Answer: siblings

63. A node with at least one child is called an _____ node.

✔️ Answer: internal

64. The _____ of a node is the number of edges from the root down to that node.

✔️ Answer: depth

65. A tree with N nodes has exactly _____ edges.

✔️ Answer: N-1

66. In a _____ Binary Tree, all leaf nodes are at the same level and every internal node has 2 children.

✔️ Answer: perfect

67. A _____ Binary Tree is shaped like a linked list, with every node having only one child.

✔️ Answer: degenerate / skewed

68. In a Balanced Binary Tree, the height difference between left and right subtrees is at most _____.

✔️ Answer: 1

69. A binary tree node in memory stores three fields: left address, _____, and right address.

✔️ Answer: data

70. File systems on a computer are a real-world example of a _____ data structure.

✔️ Answer: tree

Section E – Short Answer Questions

Write a brief but clear answer for each question (2–5 sentences).

1. What is the difference between a primitive and a non-primitive data type? Give one example of each.

Answer: A primitive data type stores a single value and is built into the language (e.g., int stores whole numbers). A non-primitive (abstract) data type is user-defined or library-defined and can store multiple values of potentially different types (e.g., Array stores a collection of integers). Primitive types have fixed memory sizes; non-primitive types can vary.

2. Why is a base case necessary in recursion?

Answer: A base case is the condition that stops a recursive method from calling itself. Without a base case, the method would call itself indefinitely, consuming stack memory until a StackOverflowError occurs. The base case represents the simplest form of the problem that can be solved directly without further recursion.

3. Compare arrays and linked lists. When would you choose a linked list over an array?

Answer: Arrays have a fixed size declared at creation and store elements in contiguous memory, allowing fast random access by index (O(1)). Linked lists have a dynamic size; nodes are scattered in memory and linked by pointers, so access requires traversal (O(n)). Choose a linked list when the number of elements is unknown in advance, or when frequent insertions and deletions in the middle of the collection are needed.

4. What is the difference between the depth and height of a node in a tree?

Answer: The DEPTH of a node is the number of edges from the ROOT down to that node (root depth = 0). The HEIGHT of a node is the number of edges on the LONGEST PATH from that node DOWN to any leaf (leaf height = 0). The height of the entire tree equals the height of the root node.

5. Explain the difference between a Full Binary Tree and a Complete Binary Tree.

Answer: A Full (strict) Binary Tree requires every node to have exactly 0 or 2 children – no node may have exactly 1 child. A Complete Binary Tree requires all levels to be fully filled except possibly the last level, which must be filled from left to right. A full binary tree does not have to be complete, and a complete binary tree does not have to be full.

6. What is the role of a constructor in Java? How is it different from a regular method?

Answer: A constructor initialises a newly created object by assigning initial values to its fields. It is called automatically when 'new ClassName()' is executed. Unlike a regular method, a constructor: (1) must have the same name as its class, (2) has no return type (not even void), and (3) cannot be declared static, abstract, final, or synchronized.

7. Describe one real-world application of each: (a) a tree data structure, (b) a queue.

Answer: (a) Tree: A computer file system uses a tree to organise folders and files hierarchically. Each folder is a node; subfolders and files are its children. You navigate from root (C:\) down to any file. (b) Queue: A print spooler uses a queue. Documents sent to a printer are printed in the order received – the first document sent is the first printed (FIFO).

8. What does it mean to say Bubble Sort has a comparison-based approach?

Answer: Comparison-based means Bubble Sort determines the correct order of elements by directly comparing their values. It compares adjacent pairs and swaps them if the left element is greater than the right. The process repeats over multiple passes until no swaps are needed, indicating the array is sorted. It does not use any arithmetic properties of the values (unlike Radix Sort).

Section F – Practical / Coding Questions

Write Java programs or trace algorithms for the following problems.

Question 1 – Array Operations

Given the following array: int A[] = {55, 20, 10, 45, 30};

Write a Java program that:

7. Stores the elements above in an array named A.
8. Displays the element at index 3.
9. Updates the element at index 0 to 99.
10. Displays all elements using a for-loop.

Solution:

```
public class ArrayOps {
    public static void main(String[] args) {

        // 1. Store elements
        int A[] = { 55, 20, 10, 45, 30 };

        // 2. Display element at index 3
        System.out.println("Element at index 3: " + A[3]);  // Output: 45

        // 3. Update element at index 0
        A[0] = 99;

        // 4. Display all elements
        System.out.println("All elements after update:");
        for (int i = 0; i < A.length; i++) {
            System.out.println("A[" + i + "] = " + A[i]);
        }
    }
}

/* Expected Output:
   Element at index 3: 45
   All elements after update:
   A[0] = 99
   A[1] = 20
   A[2] = 10
   A[3] = 45
   A[4] = 30
*/
```

Question 2 – Recursive Fibonacci

Write a recursive Java method that returns the nth Fibonacci number. The Fibonacci sequence: 0, 1, 1, 2, 3, 5, 8, 13, 21, ... (each number = sum of the two before it).

Solution:

```
public class Fibonacci {

    static int fib(int n) {
        if (n == 0) return 0;           // Base case 1
        if (n == 1) return 1;           // Base case 2
        return fib(n - 1) + fib(n - 2); // Recursive case
    }

    public static void main(String[] args) {
        for (int i = 0; i <= 8; i++) {
            System.out.print(fib(i) + " ");
        }
        // Output: 0 1 1 2 3 5 8 13 21
    }
}
```

Question 3 – Bubble Sort Trace

Trace Bubble Sort on the array: {50, 20, 40, 10, 30}. Show each pass.

Solution:

Pass	Comparison	Action	Array State
Pass 1	50 vs 20	SWAP	[20,50,40,10,30]
Pass 1	50 vs 40	SWAP	[20,40,50,10,30]
Pass 1	50 vs 10	SWAP	[20,40,10,50,30]
Pass 1	50 vs 30	SWAP	[20,40,10,30,50]
Pass 2	20 vs 40	No Swap	[20,40,10,30,50]
Pass 2	40 vs 10	SWAP	[20,10,40,30,50]
Pass 2	40 vs 30	SWAP	[20,10,30,40,50]
Pass 3	20 vs 10	SWAP	[10,20,30,40,50]
Pass 3	20 vs 30	No Swap	[10,20,30,40,50]
Pass 4	10 vs 20	No Swap	Sorted! [10,20,30,40,50]

Question 4 – Java Class & Object

Create a Java class called 'Car' with the following: Instance variables: brand (String), speed (int), colour (String). A constructor that initialises all three variables. A method displayInfo() that prints all details. In main(), create two Car objects and call displayInfo() on each.

Solution:

```
public class Car {

    // Instance variables
    String brand;
    int    speed;
    String colour;

    // Constructor
    Car(String b, int s, String c) {
        brand  = b;
        speed  = s;
        colour = c;
    }

    // Display method
    public void displayInfo() {
        System.out.println("Brand:  " + brand);
        System.out.println("Speed:  " + speed + " km/h");
        System.out.println("Colour: " + colour);
        System.out.println("-------------------");
    }

    public static void main(String[] args) {
        Car car1 = new Car("Toyota", 120, "Red");
        Car car2 = new Car("BMW",    200, "Black");
        car1.displayInfo();
        car2.displayInfo();
    }
}

/* Expected Output:
   Brand:  Toyota   Speed: 120 km/h   Colour: Red
   -------------------
   Brand:  BMW      Speed: 200 km/h   Colour: Black
   -------------------
*/
```

Question 5 – Sequential Search

Write a Java program to search for the number 75 in the array: {12, 45, 75, 23, 89, 60}. Print whether it was found and at which index.

Solution:

```
public class Search {

    static int findIndex(int A[], int K) {
        for (int i = 0; i < A.length; i++) {
            if (A[i] == K) {
                return i;    // Return the index where K was found
            }
        }
        return -1;           // K not found
    }

    public static void main(String[] args) {
        int A[] = { 12, 45, 75, 23, 89, 60 };
        int K = 75;
        int result = findIndex(A, K);

        if (result != -1) {
            System.out.println(K + " found at index " + result);
        } else {
            System.out.println(K + " not found in the array.");
        }
        // Output: 75 found at index 2
    }
}
```

Quick Reference – Data Structures at a Glance

Structure	Type	Key Property	Common Operations
Array	Linear / Static	Fixed size; contiguous memory; O(1) access by index.	Access, Update, Traverse, Sort, Search
Stack	Linear / Dynamic	LIFO – last element added is first removed.	Push, Pop, Peek
Queue	Linear / Dynamic	FIFO – first element added is first removed.	Enqueue, Dequeue, Peek
Linked List	Linear / Dynamic	Dynamic size; nodes linked by pointers; no contiguous memory.	Insert, Delete, Display, Search
Tree	Non-linear	Hierarchical; parent-child relationships across levels.	Traversal, Insert, Delete, Search
Binary Tree	Non-linear	At most 2 children per node; left and right subtrees.	Traversal (In/Pre/Post), Insert, Search
Graph	Non-linear	Nodes (vertices) connected by edges; can be cyclic.	BFS, DFS, Shortest Path

End of Data Structures & Algorithms